

BAGIAN 2 DARI 2 • PERTEMUAN 3

Terminologi dan Teknik Dasar (2)

Optimisasi, Prediksi, dan Evaluasi Model

Cakupan materi ini

- ① Optimisasi — algoritma iteratif dan penyetelan hyperparameter
- ② Prediksi dan Evaluasi — melakukan inferensi serta mengukur keberhasilan

Tujuan Pembelajaran

Setelah materi ini, mahasiswa diharapkan mampu:

1

Menjelaskan

Menerangkan cara kerja algoritma optimisasi iteratif dan peran fungsi kerugian di dalamnya.

2

Membedakan

Memisahkan parameter yang dipelajari model dari hyperparameter yang kita tentukan sendiri.

3

Menyetel

Melakukan penyetelan hyperparameter dengan data validasi serta membaca kurva latih–validasi.

4

Mengevaluasi

Memilih metrik evaluasi yang sesuai dan menafsirkan confusion matrix beserta turunannya.

Posisi Kita dalam Siklus Hidup ML

Dua tahap awal sudah dibahas. Sekarang kita melatih dan menilai modelnya.



Siklus ini jarang berjalan lurus. Hasil evaluasi hampir selalu mengirim kita kembali ke tahap perancangan model atau bahkan ke perumusan masalah.

BAGIAN 01

Optimisasi

Menemukan nilai parameter terbaik di dalam ruang hipotesis.

- 1. Optimisasi
- 2. Prediksi dan Evaluasi

Apa yang Sebenarnya Dicari Saat Melatih?

Melatih model berarti mencari satu titik terbaik di dalam ruang hipotesis.

1

Ruang hipotesis

Berisi semua model yang mungkin, misalnya seluruh garis lurus dengan berbagai kemiringan dan titik potong.

2

Fungsi kerugian

Memberi satu angka untuk setiap kandidat model: seberapa besar kesalahannya pada data latih.

3

Optimisasi

Mencari kombinasi parameter yang membuat angka kerugian itu sekecil mungkin.

Analogi: menuruni bukit dalam kabut

Bayangkan Anda berdiri di lereng bukit dan ingin mencapai titik terendah, tetapi kabut tebal membuat Anda hanya bisa merasakan kemiringan tanah di bawah kaki. Strateginya: melangkah sedikit demi sedikit ke arah yang paling menurun, lalu ulangi. Itulah inti algoritma optimisasi iteratif.

Fungsi Kerugian (Loss Function)

Kompas yang menuntun proses pelatihan.

Untuk regresi

- Mean Squared Error (MSE): rata-rata kuadrat selisih prediksi dan nilai sebenarnya.
- Menghukum kesalahan besar jauh lebih berat.
- Mean Absolute Error (MAE) lebih tahan terhadap pencilan.

Untuk klasifikasi

- Cross-entropy: menghukum model yang memberi probabilitas rendah pada kelas yang benar.
- Semakin percaya diri tetapi salah, semakin besar hukumannya.
- Bekerja pada probabilitas, bukan pada label akhir.

Syarat sebuah fungsi kerugian yang baik

Nilainya turun ketika prediksi membaik, dan bentuknya cukup mulus sehingga kemiringannya dapat dihitung. Karena itulah akurasi tidak dipakai sebagai fungsi kerugian: nilainya melompat-lompat dan tidak memberi arah perbaikan.

Algoritma Optimisasi Iteratif

Perbaiki sedikit demi sedikit, diulang ribuan kali.

1

Mulai dari tebakan awal

Parameter model diberi nilai awal, biasanya acak kecil atau nol.

2

Hitung kesalahan

Model memprediksi data latih, lalu fungsi kerugian menghitung seberapa jauh melesetnya.

3

Hitung arah perbaikan

Gradien menunjukkan ke arah mana parameter harus digeser agar kerugian mengecil.

4

Perbarui parameter

Parameter digeser sedikit ke arah tersebut, sebesar laju pembelajaran.

5

Ulangi

Langkah 2–4 diulang sampai kerugian berhenti membaik atau batas iterasi tercapai.

Laju pembelajaran

Seberapa besar langkah yang diambil pada setiap iterasi.

Terlalu kecil

Pelatihan sangat lambat dan bisa berhenti sebelum mencapai titik terbaik.

Terlalu besar

Langkahnya melompati titik terendah; kerugian naik-turun atau bahkan meledak.

Pas

Kerugian turun stabil lalu mendatar.

Varian Gradient Descent

Perbedaannya hanya pada berapa banyak data yang dipakai untuk menghitung satu langkah.

Seluruh data latih

Batch Gradient Descent

- Arah langkah paling akurat dan stabil.
- Sangat lambat bila datanya besar.

Satu contoh per langkah

Stochastic (SGD)

- Sangat cepat memperbarui parameter.
- Arahnya berisik dan naik-turun.

Sekelompok kecil contoh

Mini-batch

- Kompromi terbaik antara keduanya.
- Paling banyak dipakai dalam praktik.

Catatan: banyak model klasik di Scikit-Learn (regresi linear, SVM kecil) diselesaikan dengan metode tertutup atau pemecah khusus, bukan gradient descent. Gradient descent menjadi wajib ketika modelnya besar seperti pada deep learning.

Parameter vs Hyperparameter

Perbedaan mendasar yang menentukan cara menyetel keduanya.

Aspek	Parameter	Hyperparameter
Siapa yang menentukan	Dipelajari sendiri oleh model dari data	Ditetapkan manusia sebelum pelatihan
Kapan ditetapkan	Selama proses pelatihan	Sebelum pelatihan dimulai
Contoh	Bobot dan bias regresi, aturan pemisah pada pohon	Laju pembelajaran, kekuatan regularisasi, kedalaman pohon, nilai k pada k-NN
Cara menyetel	Algoritma optimisasi	Percobaan berulang, dinilai dengan data validasi
Data yang dipakai	Data latih	Data validasi

Cara mengingatnya: parameter adalah hasil belajar model; hyperparameter adalah pengaturan yang kita putar sendiri sebelum model mulai belajar.

Penyetelan Hyperparameter: Grid Search

Mencoba semua kombinasi nilai pada sebuah kisi, lalu memilih yang terbaik di data validasi.

```
for hp1 in [0.1, 1, 10]:  
    for hp2 in [0.1, 1, 10]:  
        model = MyModel(hp1=hp1, hp2=hp2)  
        model.fit(X_train, y_train)  
        err[hp1, hp2] = error(y_val,  
                               model.predict(X_val))  
  
hp1_best, hp2_best = arg_min(err)
```

Galat pada data validasi

hp1 \ hp2	0.1	1	10
0.1	.8	.7	.7
1	.4	.2	.3
10	.5	.6	.6

Kombinasi terbaik: hp1 = 1 dan hp2 = 1, dengan galat validasi 0,2.

Keterbatasan grid search

Jumlah percobaan tumbuh sangat cepat: tiga hyperparameter dengan lima pilihan nilai berarti 125 kali pelatihan. Alternatifnya adalah random search, yang sering menemukan hasil setara dengan percobaan jauh lebih sedikit, serta optimisasi Bayesian untuk pencarian yang lebih terarah.

Validasi Silang (Cross-Validation)

Solusi ketika data terlalu sedikit untuk disisihkan sebagai data validasi.

Cara kerja k-fold

- Data latih dibagi menjadi k bagian yang sama besar, misalnya $k = 5$.
- Model dilatih k kali; setiap kali satu bagian menjadi data validasi dan sisanya menjadi data latih.
- Skor akhir adalah rata-rata dari k percobaan tersebut.

Mengapa berguna

- Seluruh data ikut dipakai untuk melatih sekaligus menilai.
- Penilaian lebih stabil, tidak bergantung pada satu pembagian yang kebetulan menguntungkan.
- Biayanya: waktu pelatihan menjadi k kali lipat.

```
from sklearn.model_selection import GridSearchCV
grid = GridSearchCV(LogisticRegression(), {'C': [0.01, 0.1, 1, 10]}, cv=5)
grid.fit(X_train, y_train); grid.best_params_
```

Akurasi Latih vs Akurasi Validasi

Grafik yang wajib Anda periksa setiap kali menyetel model.

1

Akurasi latih hampir selalu lebih tinggi

Model memang dioptimalkan pada data itu, jadi wajar bila hasilnya lebih bagus.

2

Akurasi latih terus naik seiring kompleksitas

Model yang makin rumit selalu bisa mencocokkan data latih lebih baik.

3

Akurasi validasi naik lalu turun

Awalnya model belajar pola nyata, kemudian mulai menghafal derau. Titik puncaknya adalah titik seimbang.

Regularisasi bekerja dengan cara yang sama: terlalu lemah menyebabkan overfitting, terlalu kuat menyebabkan underfitting.

Cara membacanya

- Keduanya rendah → underfitting. Perlu model lebih kuat atau fitur lebih baik.
- Latih tinggi, validasi jauh lebih rendah → overfitting. Perkuat regularisasi atau tambah data.
- Keduanya tinggi dan berdekatan → inilah titik yang dicari.

Kesalahan Umum Saat Optimisasi

Empat hal yang paling sering merusak hasil eksperimen mahasiswa.

Menyetel dengan data uji

Begitu data uji dipakai memilih hyperparameter, angkanya berhenti mengukur generalisasi.

Melakukan fit penskala pada seluruh data

Statistik dari data uji ikut bocor ke data latih (data leakage), sehingga hasil terlihat terlalu bagus.

Tidak menetapkan `random_state`

Hasil eksperimen tidak dapat diulang, sehingga perbandingan antarmodel tidak adil.

Membandingkan model dengan data latih

Model paling rumit akan selalu menang, padahal justru dialah yang paling mungkin overfitting.

Gunakan Pipeline di Scikit-Learn: seluruh langkah pra-pemrosesan dan model dibungkus jadi satu, sehingga fit hanya terjadi pada data latih di setiap lipatan validasi.

BAGIAN 02

Prediksi dan Evaluasi

Menggunakan model yang sudah dilatih, lalu menilai apakah ia benar-benar berguna.

- 1. Optimisasi
- 2. Prediksi dan Evaluasi

Inferensi: Membuat Prediksi

Inferensi adalah proses memakai model terlatih untuk memprediksi data baru.

Prediksi label

```
model.predict()
```

- Mengembalikan satu nilai: kelas atau angka hasil regresi.
- Menunjukkan nilai yang paling mungkin menurut model.
- Tidak memberi tahu seberapa yakin model tersebut.

Prediksi distribusi

```
model.predict_proba()
```

- Mengembalikan probabilitas untuk setiap kelas.
- Menyimpan informasi ketidakpastian prediksi.
- Untuk regresi: dapat berupa rata-rata beserta variansinya.

Contoh: `predict()` menjawab “Sweater”. `predict_proba()` menjawab “Sweater 62%, Kaos 25%, Jaket 13%” — jawaban kedua jauh lebih berguna bila keputusan berisiko, karena kita tahu model sebenarnya ragu.

Mengapa Ketidakpastian Itu Penting

Satu label tunggal menyembunyikan informasi yang sering justru paling menentukan.

1

Menentukan ambang keputusan

Dengan probabilitas, kita bebas memilih ambang batas sesuai risiko — misalnya baru menandai email sebagai spam bila keyakinan di atas 95%.

2

Menyerahkan kasus sulit ke manusia

Prediksi dengan keyakinan rendah dapat dialihkan ke peninjau manusia, bukan diputuskan otomatis.

3

Mengurutkan berdasarkan keyakinan

Berguna untuk sistem rekomendasi dan pencarian, yang butuh peringkat, bukan sekadar label.

4

Dasar teknik modern

Banyak model masa kini, termasuk model bahasa besar, pada dasarnya memprediksi distribusi peluang.

Karena itu, biasanya memeriksa `predict_proba()` sebelum menyimpulkan bahwa model sudah bekerja dengan baik.

Akurasi dan Keterbatasannya

Metrik paling populer, sekaligus paling sering menyesatkan.

Definisi

Akurasi = jumlah prediksi benar ÷ jumlah seluruh prediksi.

Mudah dipahami dan cocok bila jumlah data tiap kelas relatif seimbang.

Masalah kelas timpang

Bila hanya 1% transaksi adalah penipuan, model yang selalu menjawab “bukan penipuan” mencapai akurasi 99% — tetapi sama sekali tidak berguna.

Apakah semua jenis kesalahan setara?

- Menandai email penting sebagai spam berbeda akibatnya dengan meloloskan satu email spam ke kotak masuk.
- Melewatkan diagnosis penyakit jauh lebih berbahaya daripada memberi rujukan pemeriksaan tambahan yang ternyata tidak perlu.
- Karena itu kita perlu mengukur biaya setiap jenis kesalahan, bukan hanya menghitung jumlah kesalahan.

Confusion Matrix

Membongkar akurasi menjadi empat jenis hasil yang berbeda maknanya.

Contoh: deteksi spam	Sebenarnya Spam	Sebenarnya Bukan Spam
Diprediksi Spam	True Positive (TP) Benar ditandai spam	False Positive (FP) Email penting ikut ditandai
Diprediksi Bukan Spam	False Negative (FN) Spam lolos ke kotak masuk	True Negative (TN) Benar dilewatkan

Cara membacanya: kata pertama menyatakan apakah prediksi model benar (True/False); kata kedua menyatakan apa yang diprediksi model (Positive/Negative). Jadi False Negative berarti model menjawab “negatif” dan jawaban itu salah.

Precision, Recall, dan F1

Tiga metrik yang menjawab pertanyaan berbeda.

Precision

$$TP \div (TP + FP)$$

Dari semua yang ditandai positif, berapa yang benar-benar positif?

Dipakai bila false positive mahal — misalnya email penting masuk folder spam.

Recall

$$TP \div (TP + FN)$$

Dari semua yang sebenarnya positif, berapa yang berhasil ditemukan?

Dipakai bila false negative berbahaya — misalnya melewatkan diagnosis penyakit.

F1-Score

Rata-rata harmonik

Menggabungkan precision dan recall menjadi satu angka.

Dipakai saat keduanya sama-sama penting dan kelasnya timpang.

Precision dan recall saling bertukar: menaikkan ambang keputusan menaikkan precision tetapi menurunkan recall, dan sebaliknya.

Metrik untuk Masalah Regresi

Label kontinu menuntut ukuran kesalahan yang berbeda.

MAE

Mean Absolute Error

Rata-rata besar selisih prediksi dan nilai sebenarnya. Satuannya sama dengan target, sehingga mudah dijelaskan. Cukup tahan terhadap pencilan.

MSE / RMSE

Mean Squared Error

Menghukum kesalahan besar jauh lebih berat. RMSE mengembalikan satuannya ke satuan target.

R^2

Koefisien determinasi

Seberapa besar variasi target yang berhasil dijelaskan model. Bernilai 1 bila sempurna, 0 bila setara dengan sekadar menebak rata-rata.

Sama seperti klasifikasi: pilih metrik berdasarkan konsekuensi kesalahannya, bukan berdasarkan mana yang paling sering muncul di tutorial.

Alur Lengkap dengan Scikit-Learn

Semua yang sudah dibahas, dirangkai menjadi satu berkas kode.

```
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix

X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=42)

pipe = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
grid = GridSearchCV(pipe, {'logisticregression__C': [0.01, 0.1, 1, 10]}, cv=5)
grid.fit(X_tr, y_tr)                                # penyetelan pakai data latih saja

y_pred = grid.predict(X_te)                        # data uji dipakai SEKALI di sini
print(confusion_matrix(y_te, y_pred))
print(classification_report(y_te, y_pred))
```

Pipeline

mencegah kebocoran data

GridSearchCV

menyetel dengan validasi silang

classification_report

precision, recall, F1 sekaligus

Ringkasan Materi Ini

Enam gagasan kunci dari tahap Optimisasi serta Prediksi dan Evaluasi.

1

Melatih model berarti mencari parameter yang meminimalkan fungsi kerugian.

2

Algoritma iteratif bekerja dengan langkah kecil berulang; laju pembelajaran menentukan besar langkahnya.

3

Parameter dipelajari dari data latih; hyperparameter disetel memakai data validasi.

4

Grid search, random search, dan validasi silang adalah cara baku menyetel hyperparameter.

5

`predict()` memberi satu label, `predict_proba()` memberi tingkat keyakinan yang sering lebih berguna.

6

Akurasi saja tidak cukup: gunakan confusion matrix, precision, recall, dan F1 sesuai risiko kesalahannya.

Tugas dan Latihan Mandiri

Menggabungkan seluruh materi Pertemuan 3.

Tugas praktikum

- Pilih satu dataset klasifikasi, lalu rumuskan target, tujuan, dan datanya secara tertulis.
- Bagi data menjadi latih, validasi, dan uji dengan `random_state` tetap.
- Bangun Pipeline berisi pra-pemrosesan dan model linear sebagai baseline.
- Setel minimal satu hyperparameter dengan `GridSearchCV` dan validasi silang.
- Laporkan confusion matrix beserta precision, recall, dan F1 pada data uji.

Pertanyaan untuk direnungkan

- Pada kasus Anda, mana yang lebih merugikan: false positive atau false negative? Metrik mana yang seharusnya dipakai?
- Apa yang terjadi pada akurasi latih dan validasi ketika regularisasi diperkuat terus-menerus?
- Mengapa penskalaan fitur harus berada di dalam Pipeline, bukan dijalankan lebih dulu pada seluruh data?
- Kapan sebuah model dengan akurasi 99% justru patut dicurigai?

TERIMA KASIH

Terminologi dan Teknik Dasar (2)

Siklus hidup ML kini lengkap: masalah, model, optimisasi, dan evaluasi.

Materi diadaptasi dari kuliah “Machine Learning Mechanics — Terminology and Techniques” oleh Joseph E. Gonzalez dan Narges Norouzi. Rujukan buku: Bab 1.